UNIVERSITY OF WOLVERHAMPTON
KNOWLEDGE ▪ INNOVATION ▪ ENTERPRISE

6CS014 Complex System

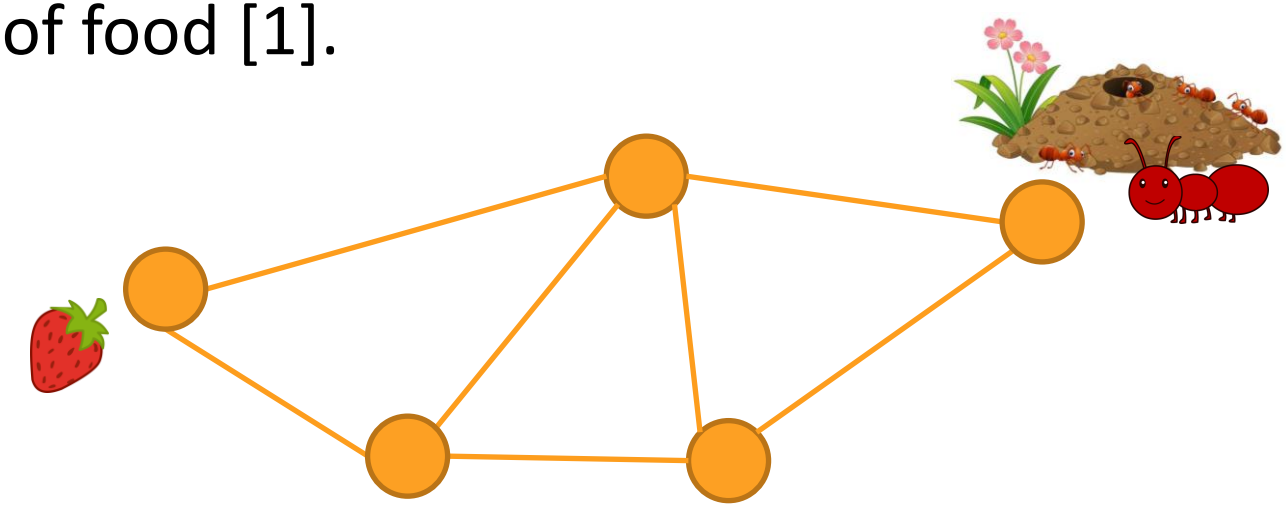
Ant Colony Optimisation (ACO) Overview

Learning Outcomes

- By the end of this session you will be able to:
  - appraise the fundamentals of the ACO algorithm.

Introduction

- **Ant Colony Optimisation (ACO) searches for an optimal path in a graph.**
- It does this based on behaviour of ants seeking a path between their colony and a source of food [1].
- Probabilistic technique.

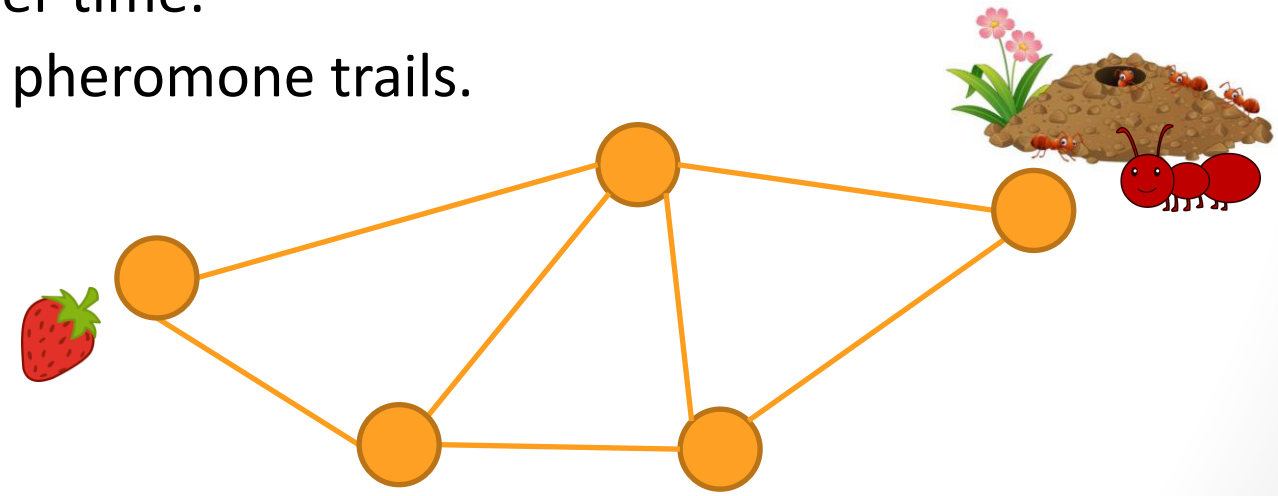


Example of ants searching for food.

Ant image source:
https://webstockreview.net/image/ant-clipart-red-ant/567749.html
Strawberry image source:
https://www.pinterest.com/pin/232850243210428406/
Ants nest image source:
https://www.dreamstime.com/stock-illustration-cartoon-ants-colony-ant-hill-illustration-image77834495

Overview of the Concept

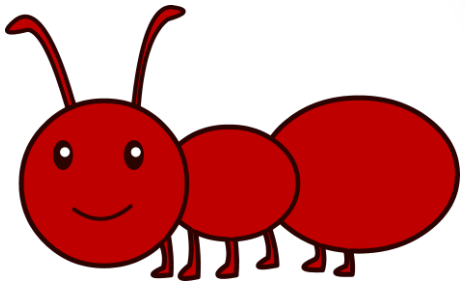
- **Overview of concept (how ants behave in the real world) [1]:**
  - Ants navigate from nest to food source.
  - Ant leave a pheromone as they travel down a path.
  - More pheromone on path increases probability of path being followed by other ants.
  - The pheromone evaporates over time.
  - Shortest path is discovered via pheromone trails.

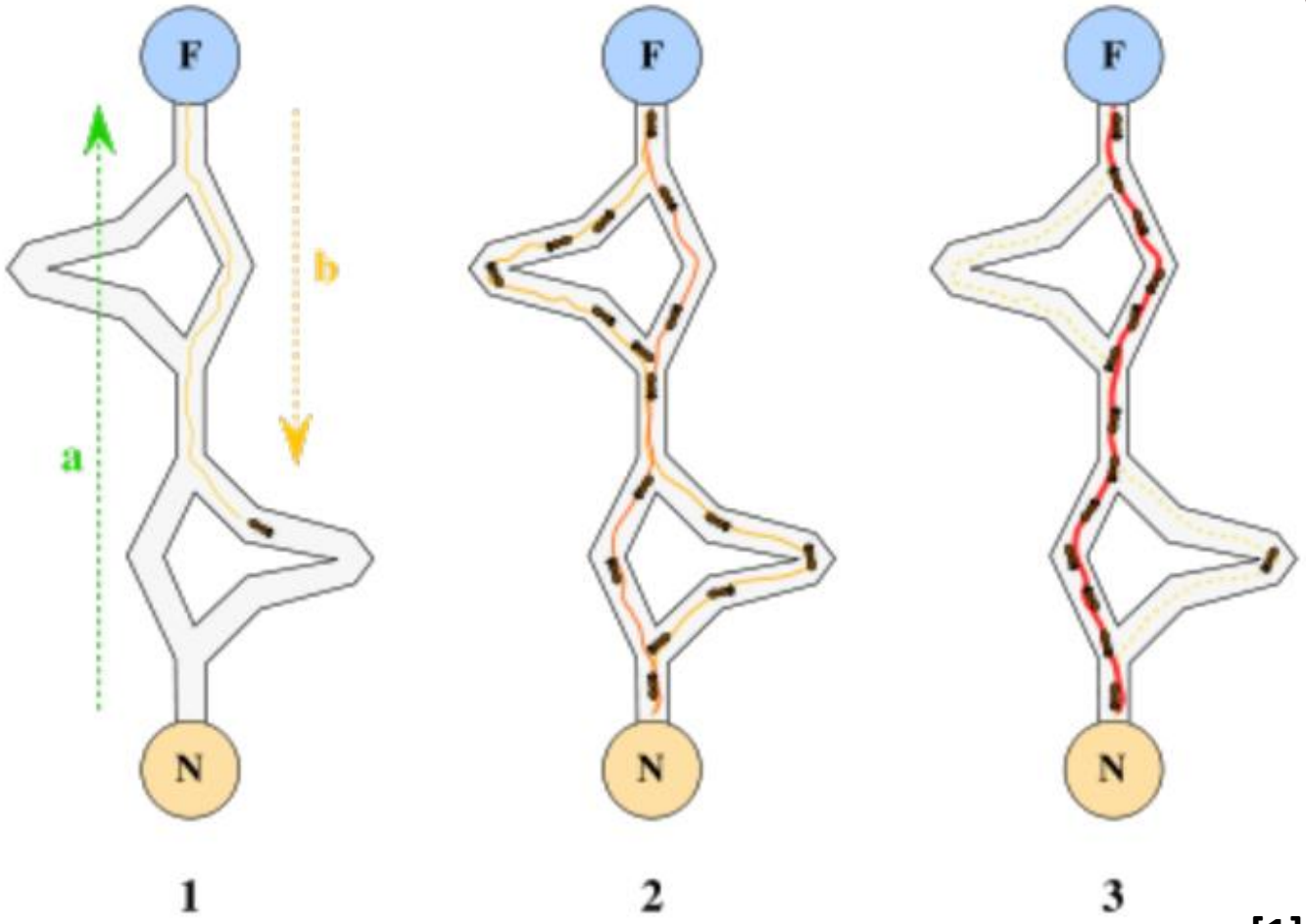


Example of ants searching for food.

Ant image source:
https://webstockreview.net/image/ant-clipart-red-ant/567749.html
Strawberry image source:
https://www.pinterest.com/pin/232850243210428406/
Ants nest image source:
https://www.dreamstime.com/stock-illustration-cartoon-ants-colony-ant-hill-illustration-image77834495

Overview of the Concept





[1]

Ant image source:
https://webstockreview.net/image/ant-clipart-red-ant/567749.html

Overview of ACO

- **Before we look at the ACO algorithm / process let's explore what problem we would like the ACO algorithm to solve.**

Traveling Salesman / Salesperson Problem

- **ACO solves the Traveling Salesperson Problem (TSP).**
- What is the travelling salesperson problem (TSP)?
  - “The travelling salesman problem (TSP) asks the following question: "Given a list of cities and the distances between **each** pair of cities, what is the shortest possible route that visits each city and returns to the origin city?"”
    - Source: https://en.wikipedia.org/wiki/Travelling_salesman_problem
- **In other words, a salesperson must start from one place, travel to all other cities just once and return to the original city.** The travelling salesman problem is all about finding the least cost (i.e. shortest distance) path with these conditions.

Traveling Salesman / Salesperson Problem

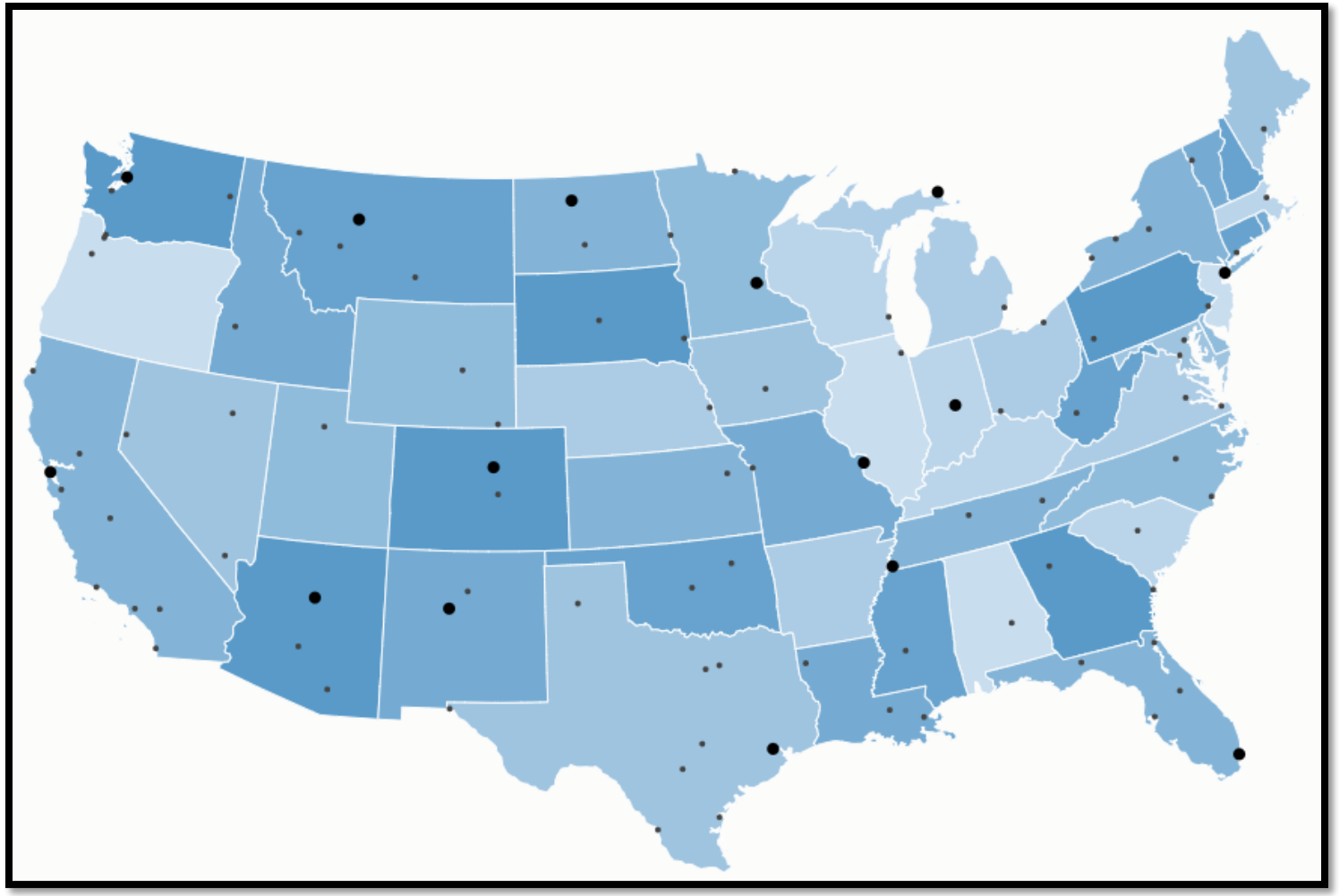


Image source: http://examples.gurobi.com/traveling-salesman-problem/

Traveling Salesman / Salesperson Problem

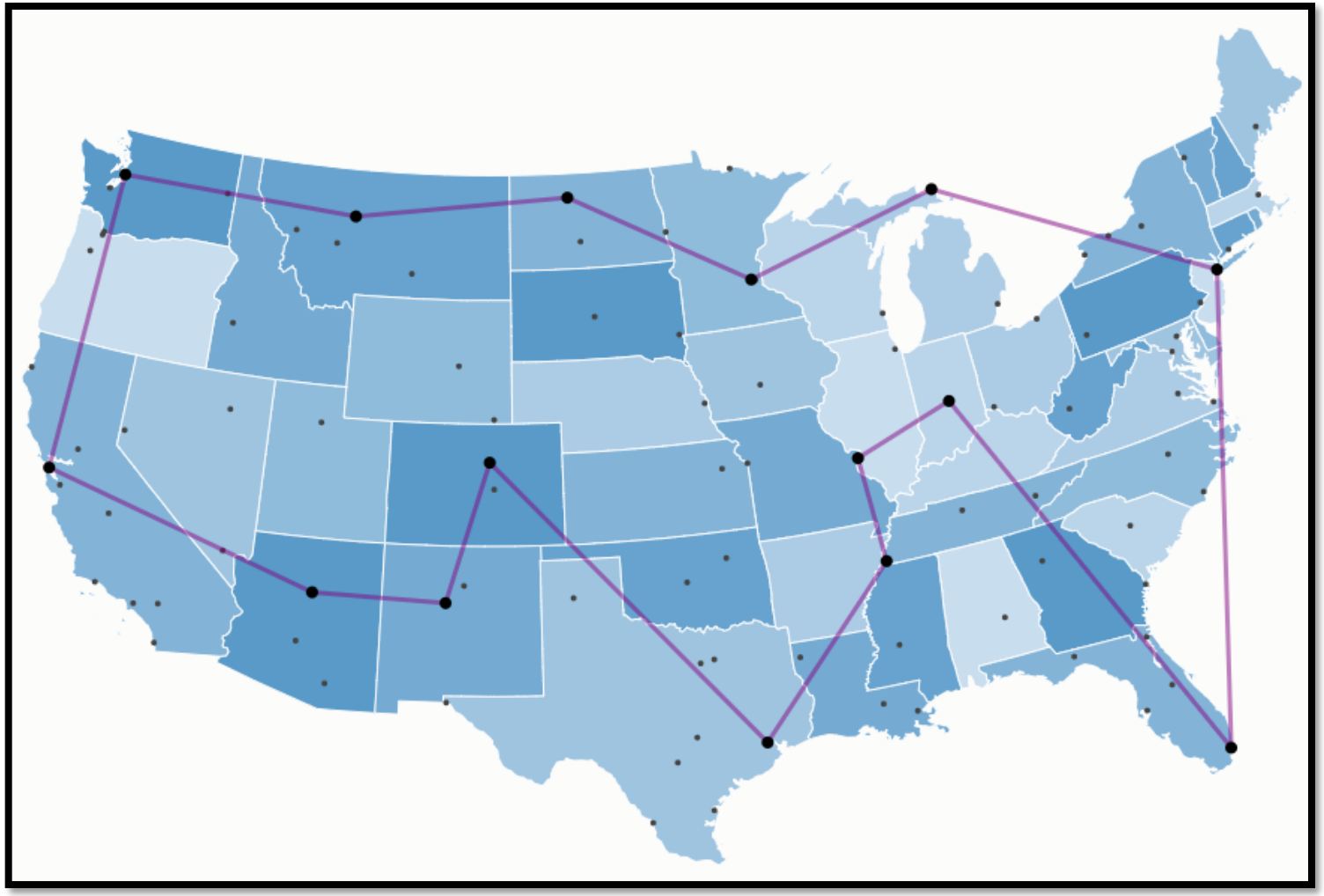


Image source: http://examples.gurobi.com/traveling-salesman-problem/

Traveling Salesman / Salesperson Problem

- A solution to this problem is simple enough.
  - Pick a starting point, determine all of the distances to the other cities and find the shortest one, repeating until you've visited them all and then return to the starting city.
  - Repeat this process for all the other combinations of cities.
  - Pick the route between all cities with the smallest distance overall. *This is the answer!*
- This is an example of a brute force approach.
- **The issue is that for even a small set of cities, the number of possible routes is massive.**

Traveling Salesman / Salesperson Problem

- Mathematically, the number of permutations for n entities is **n!** (n factorial).
- How big a number is that for various number of cities?
  - Ten Cities 10! 3,628, 800 (over three million)
  - Twenty Cities 20! 2,432,902,008,176,640,000 (over 2 quintillion)
- If we assume that a computer could compute the routes of one million cities per second:
  - For twenty cities, it would take 77,000 years.

Rough (i.e., approx.) details:

3,600 seconds/hour * 24 hours in a day = 86,400 seconds in a day.

86,400 * 365 days in a year = **31,536,000 seconds in a year (roughly).**

Thirty-one million five hundred thirty-six thousand seconds.

31,536,000 * 1 million routes per second = 31,536,000,000,000.

Thirty-one trillion, five hundred and thirty-six billion.

2,432,902,008,176,640,000 / 31,536,000,000,000 = 77,146.81 years (approx.).

Example:

3 cities:

Wolverhampton (wlv).
Birmingham (bir).
Telford (tel).

The factorial is **multiply** all the whole numbers from a number (e.g., the number of cites) down to 1.
3 x 2 x 1 = 6.

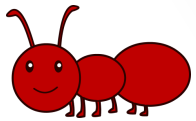
Combinations:

1 - wlv, bir, tel
2 - wlv, tel, bir
3 - bir, tel, wlv
4 - bir, wlv, tel
5 - tel, wlv, bir
6 - tel, bir, wlv

Ant Colony Optimisation (ACO)

- The Ant Colony Optimisation algorithm can be used to solve the travelling salesperson problem.
- In other words, the Ant Colony Optimisation algorithm can generate a route between all the graph nodes / cities.

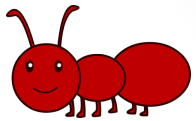
Overview of ACO



- Pheromone/trail accumulated on path segments.
  - Or connections in our graph.
- A **Path** *(i.e., a connection)* is selected at random based on amount of **pheromone** present on possible connections from the current node and the **length** of the connections.
  - I.e. If there is more pheromone along a path / connection the ants are more likely to choose that path.
  - Shorter paths are preferred over longer paths.
- Ant reaches next node, and again selects next connection / path at random based on amount of pheromone and the length of the path.
- Continues until reaches starting node again.
- Finished **tour** is a solution.
- Tour is analysed for optimality.

Ant image source:
https://webstockreview.net/image/ant-clipart-red-ant/567749.html

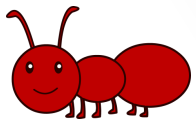
ACO – Overview of Algorithm



- If there are ***n* graph nodes or cities**, then ***l* distinct ants** start from any of these *n* nodes in the graph / cities randomly.
- These ants have two distinct advantages over real ants:
  - They have memory and do not visit the nodes / cities they have visited.
  - They know the distance between the nodes / cities and tend to choose the nearest nodes / city if the pheromones on paths are the same.
- Also, if the distance of two paths are the same, they tend to choose the path with more pheromone, like real ants.

Ant image source:
https://webstockreview.net/image/ant-clipart-red-ant/567749.html

ACO – Overview of Algorithm



- Below is pseudo code for the whole ACO algorithm.

The algorithm keeps going (i.e. looping) for a fixed number of iterations.

```
Get the coordinates of the cities
For t = 1 to iteration_threshold
    For k = 1 to l
        For move_count = 1 to n
            Let ant move to a graph node / city based on a
            probability, which is based on the connections
            phenome level and a visibility factor
        Loop
        Calculate the length of the tour for ant k
    Loop
    Update the pheromone for each connection based on a formula
Loop
```

(l = number of ants / k is an individual ant)

- The ant chooses to travel from one city to another based on a probably and other rules (*e.g. do not visit the cities they have visited*).
- The probably is based on a phenome level and a visibility factor.
- The ant selects the connection (i.e. city to travel to) with the highest probability.

- The current level of pheromone is evaporated by a factor.
- A new amount of pheromone is added if an ant travelled along the connection.
- The amount added is based on the length of the ant's tour.

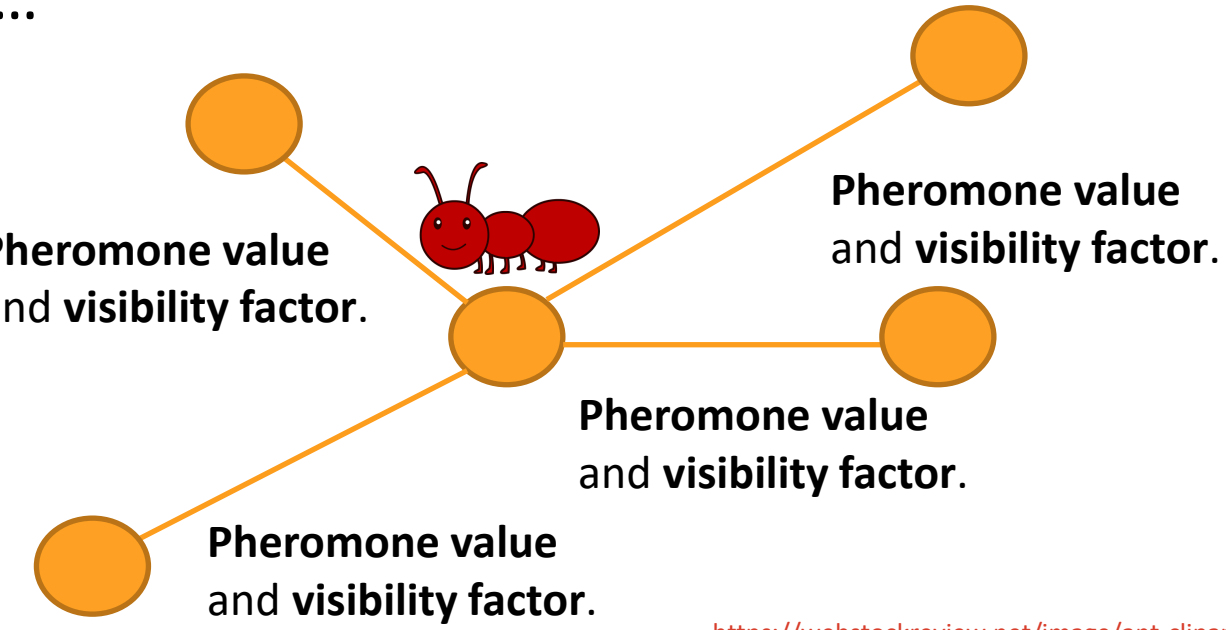
iteration_threshold is a constant value. The number of ants is also a constant value.

ACO – Overview of Algorithm: Choosing to travel to a city

- **Visibility factor:** 1 / the distance between the nodes (e.g. cities). So long distances have smaller values.

- **The probability an ant chooses to travel to graph node / city is based on the following:**
  1. Sum the product of the **pheromone level** and the **visibility factor** on all **allowed** paths from the node (e.g. city in the TSP problem) where the ant k is…

First the ant needs to work out the Total pheromone and visibility for all connections.

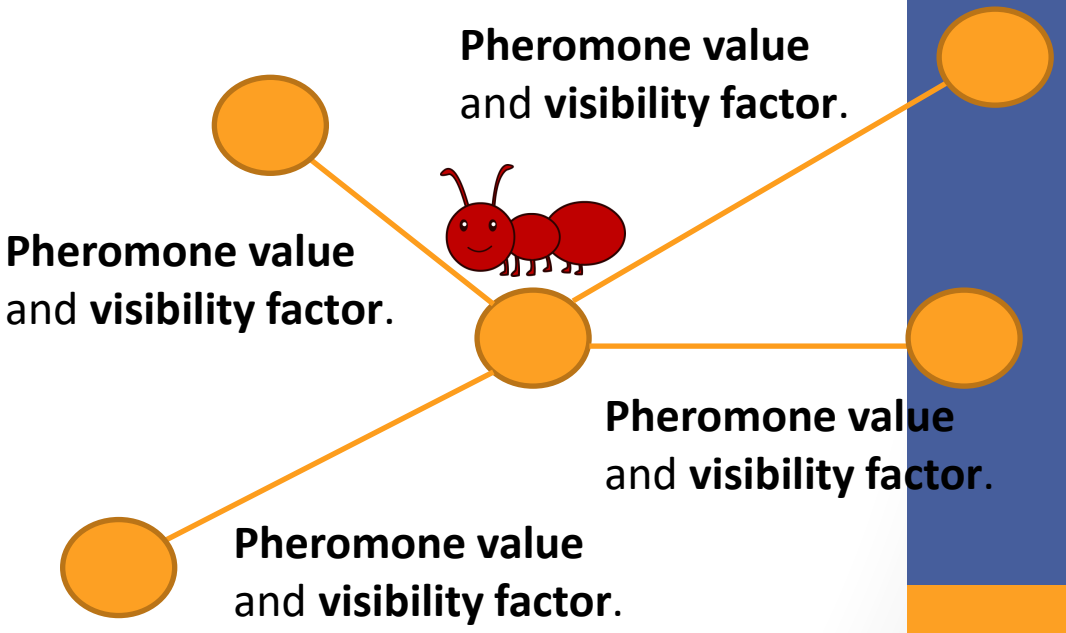


See the following equation for full details.

Ant image source: https://webstockreview.net/image/ant-clipart-red-ant/567749.html

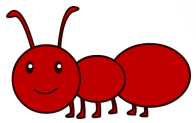
ACO – Overview of Algorithm: Choosing to travel to a city

- **The probability an ant chooses to travel to graph node / city is based on the following:**
  2. Calculate the product of the pheromone level and the visibility factor of the proposed path.
  3. Define the probability of choosing the proposed path as the value found in (2) divided by the value found in (1).
  4. Travel down the path with the largest probability - or have a random choice if there are paths with equal probabilities.



Ant image source:
https://webstockreview.net/image/ant-clipart-red-ant/567749.html

ACO – Overview of Algorithm: Choosing to travel to a city



- **The probability an ant chooses to travel to graph node / city is based on the following formula:**

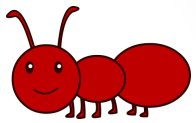
$$p_{ij}^{k} = (\tau_{ij})^{\alpha} \cdot (\eta_{ij})^{\beta} / \{ \sum_{s} (\tau_{is})^{\alpha} \cdot (\eta_{is})^{\beta} \}$$

k = ant.

- Where:
  - Where τ_{ij} is the intensity of the pheromone for connection i,j.
  - α is a parameter to regulate the influence of τ_{ij} that takes a value between $0 \leq \alpha \leq 1$ and reflects how long the pheromone lasts.
  - η_{ij} is the visibility of city *j* from city *i* and equals $1/d_{ij}$ where d_{ij} is the distance between city *i* and *j*.
  - β is a parameter to regulate the influence of η_{ij} that takes a value between $0 \leq \beta \leq 1$.

Ant image source:
https://webstockreview.net/image/ant-clipart-red-ant/567749.html

ACO – Overview of Algorithm: Choosing to travel to a city



- The previous equation can be coded in the following way.

$$p_{ij}^{k} = (\tau_{ij})^{\alpha} \cdot (\eta_{ij})^{\beta} / \{ \sum_{s} (\tau_{is})^{\alpha} \cdot (\eta_{is})^{\beta} \}$$

```
float TotalPheromoneAndVisibility = 0;

// Loop through the paths not visited and calculate total pheromone & visibility.
foreach (Connection aConnection in ConnectionsFromNodeAndNotVisited)
{
    TotalPheromoneAndVisibility += (Mathf.Pow(aConnection.GetPheromoneLevel(), Alpha) *
                                    Mathf.Pow((1 / aConnection.GetDistance()), Beta));
}
```

```
float PathProbability = (Mathf.Pow(aConnection.GetPheromoneLevel(), Alpha) *
                         Mathf.Pow((1 / aConnection.GetDistance()), Beta));

PathProbability = PathProbability / TotalPheromoneAndVisibility;
```

ACO – Overview of Algorithm: Pheromone Update

- **The Pheromone of a connection is updated on the following formula:**

$$\tau_{ij}(t+1) = (1-\rho)\cdot\tau_{ij}(t) + \Delta\tau_{ij}$$

- Where:
  - $\tau_{i,j}$ is the amount of pheromone on a given edge (i.e. connection) i, j.
  - t is the iteration counter.
  - ρ where $0 \leq \rho \leq 1$ is the evaporation factor of the pheromone (i.e. it is the rate of pheromone evaporation).
  - $\Delta\tau_{i,j}$ is the amount of pheromone deposited.

ACO – Overview of Algorithm: Pheromone Update

- $\Delta\tau_{i,j}$ is the amount of pheromone deposited. It is determined by the following:

$$\Delta\tau_{ij} = \sum_{k=1}^{l} \Delta T_{i,j}^{k}$$

recall that the number of ants = *l*

- $\Delta T_{i,j}^{k} = \begin{cases} Q/L_k \text{ if ant } k \text{ travels on edge (i, j)} \\ 0 \text{ otherwise} \end{cases}$

- Where, L_k is the length of the tour for ant *k*.
- Q is a constant, it should be ≤ 1.
- Ants whose route is longest, the value of L_k, have a smaller effect on the value of $\Delta\tau_{ij}$.

ACO – Overview of Algorithm: Pheromone Upd

```
// Update pheromone by formula Δτij.
// Loop through the paths and check if visited destination already.
foreach (Connection aConnection in Connections)
{
        float Sum = 0;
        foreach (Ant TmpAnt in Ants)
        {
            List<Connection> TmpAntConnections = TmpAnt.GetConnections();
            foreach (Connection tmpConnection in TmpAntConnections)
            {
                if (aConnection.Equals(tmpConnection))
                {
                    Sum += Q / TmpAnt.GetAntTourLength();
                }
            }
        }

        float NewPheromoneLevel = (1 - EvaporationFactor) *
                                        aConnection.GetPheromoneLevel() + Sum;

        aConnection.SetPheromoneLevel(NewPheromoneLevel);

        // Reset path probability.
        aConnection.SetPathProbability(0);
}
```

ACO – Overview of Algorithm

- **Recap** - below is pseudo code for the whole ACO algorithm.

```
Get the coordinates of the cities
For t = 1 to iteration_threshold
    For k = 1 to l
        For move_count = 1 to n
            Let ant move to a city based on a probability p_ij^k ,
            which is based on the connections phenome level
            and a visibility factor
        Loop
        Calculate L_k the length of the tour for ant k
    Loop
    Update pheromone based on a formula Δτ_ij
Loop
```

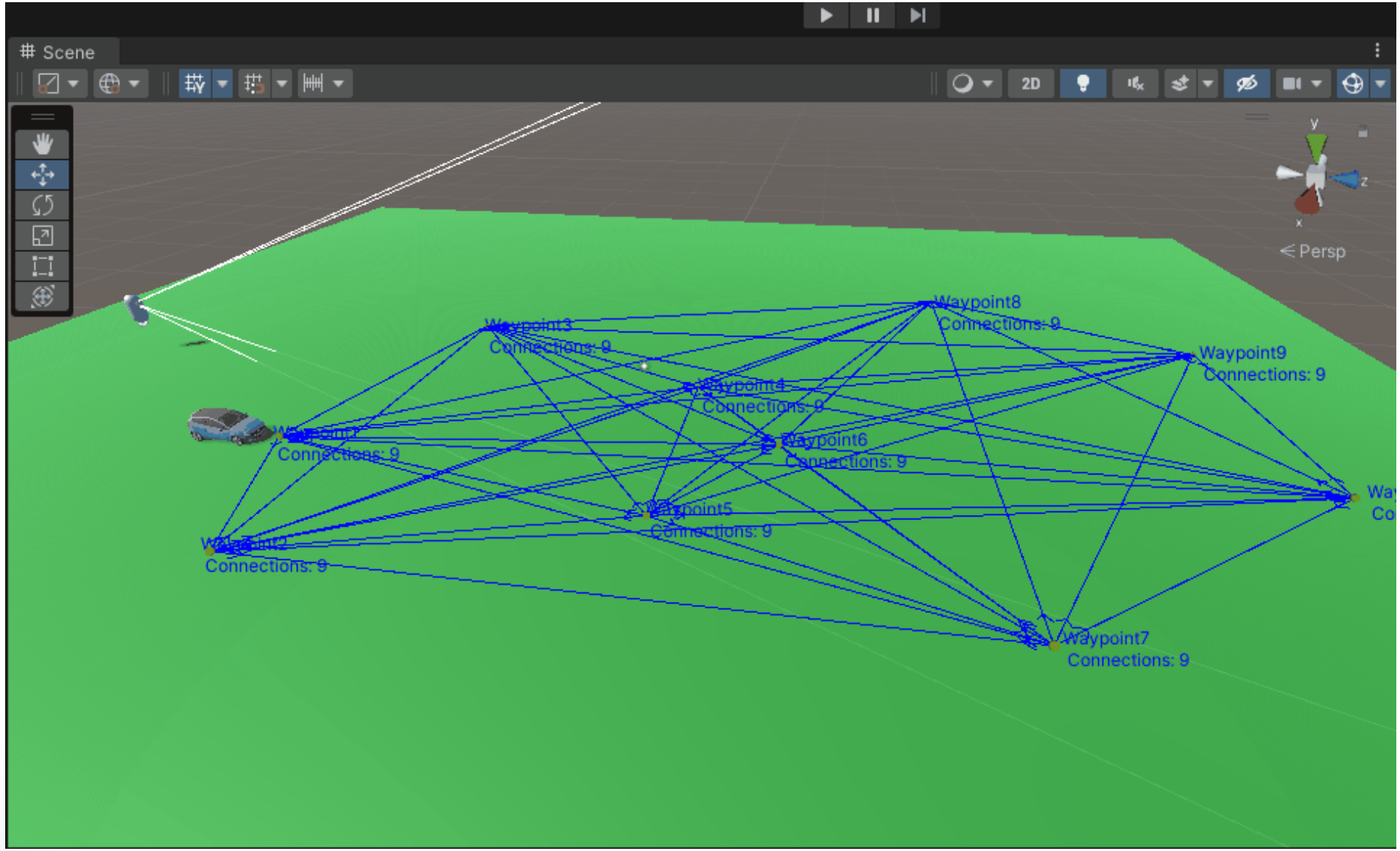
(l = number of ants / k is an individual ant)

- The ant chooses to travel from one city to another based on a probably and other rules (*e.g. do not visit the cities they have visited*).
- The probably is based on a phenome level and a visibility factor.
- The ant selects the connection (i.e. city to travel to) with the highest probability.

- The current level of pheromone is evaporated by a factor.
- A new amount of pheromone is added if the ant travelled along the connection.
- The amount added is based on the length of the tour.

iteration_threshold is a constant value. The number of ants is also a constant value.

Workbook Outcome



Summary

- By the end of this session you will be able to:
  - appraise the fundamentals of the ACO algorithm.
- **Any questions?**
- **<u>Learning Outcomes?</u>**

References

1. Ant colony Optimization Algorithms: Introduction and Beyond. Anirudh Shekhawat, Pratik Poddar, Dinesh Boswal. <http://mat.uab.cat/~alseda/MasterOpt/ACO_Intro.pdf>.